

Implementación de Trunk-Based

Development,

Pipelines CI/CD y GitHub Rulesets

Guía Técnica de Implementación, Automatización de Pruebas y Evidencias

Reales del Proceso

Estudiante/Autor: Jhardiher Jos  Giraldo Mu oz (MunGZar)

Proyecto Principal: SecurityMZManager

Repositorio Pr ctica: juanpablogarcia2020/bd

Metodolog a: Scrum + Trunk-Based Development

Plataformas: Git, GitHub Actions, GHCR (Docker)

Fecha: Septiembre 2026

Tabla de Contenidos

- 1. [Resumen Ejecutivo y Objetivos](#)

- 2. [Fundamentos de Trunk-Based Development y CI/CD](#)

- 3. [Arquitectura de Pipeline y Desacoplamiento de Jobs](#)

- 4. [Fase 1: Reestructuraci3n y Refactorizaci3n del Pipeline de CI/CD \(Capturas 1 a 9\)](#)

- 5. [Fase 2: Configuraci3n de Gobernanza con GitHub Rulesets \(Capturas 10 a 18\)](#)

- 6. [Fase 3: Verificaci3n del Ciclo TBD con Pull Request en Vivo \(Capturas 19 a 30\)](#)

- 7. [Mapeo y Aplicaci3n al Proyecto Real \(SecurityMZManager\)](#)

- 8. [Conclusiones y Buenas Pr3cticas de Ingenier3a](#)

1. Resumen Ejecutivo y Objetivos

El presente informe documenta de manera exhaustiva el desarrollo e implementación del **Laboratorio de Trunk-Based Development (TBD)**, una de las prácticas nucleares de la ingeniería de software moderna y la cultura DevOps dentro de marcos ágiles como Scrum.

El objetivo primordial de esta práctica es erradicar el fenómeno conocido como *Merge Hell* (infierno de fusiones) que padecen los equipos cuando utilizan estrategias tradicionales como GitFlow, sustituyéndolo por un flujo de trabajo basado en la rama principal (**main** o trunk) con ramas de soporte de vida ultra-corta, respaldadas por un pipeline automatizado de **Integración Continua (CI)** y **Entrega Continua (CD)**, y custodiado por políticas de **GitHub Rulesets / Branch Protection**.

Metas Técnicas Cumplidas en el Laboratorio:

- **Desacoplamiento Estricto de Jobs:** Separar la ejecución de pruebas unitarias/análisis estático (**test**) del proceso de empaquetado y subida a contenedores (**build_and_push**).
- **Principio de Mínimo Privilegio:** Limitar los permisos de escritura (**packages: write**) exclusivamente al job de publicación en **main**, manteniendo los Pull Requests en solo lectura.
- **Gobernanza con GitHub Rulesets:** Prohibir el push directo a la rama troncal, exigir al menos una revisión aprobada y requerir que los checks automatizados pasen satisfactoriamente antes de cualquier fusión.
- **Validación Empírica del Ciclo Completo:** Creación de rama efímera, simulación de cambio atómico, verificación de bloqueo en PR, ejecución de checks, fusión mediante *Squash and Merge* y activación automática del empaquetado final a GitHub Container Registry (GHCR).
- **Higiene de Ramas (Branch Hygiene):** Comprensión técnica de por qué nunca se debe eliminar una rama de desarrollo hasta que la interfaz confirme el estado *Merged*.

2. Fundamentos de Trunk-Based Development y CI/CD

En el desarrollo de software convencional bajo GitFlow, los desarrolladores suelen trabajar en ramas de características (*feature branches*) que permanecen abiertas durante semanas. Cuando múltiples miembros del equipo intentan integrar sus ramas simultáneamente hacia **develop** o **main**, surgen

conflictos de código masivos, divergencias de dependencias y regresiones inadvertidas.

Trunk-Based Development (TBD) invierte este paradigma: todos los desarrolladores integran cambios pequeños, atómicos y frecuentes directamente en una única rama troncal (**main**), típicamente varias veces al día. Para mitigar cualquier riesgo de inestabilidad, la rama principal es blindada mediante:

1. **Ramas de Vida Corta (Short-Lived Branches):** Ramas que duran desde unas pocas horas hasta máximo 1 o 2 días.
2. **Validación Automática Inmediata (CI Gate):** Cada Pull Request ejecuta de forma obligatoria linters y suites de pruebas unitarias. Si una sola prueba falla, la integración es rechazada de inmediato.
3. **Feature Flags / Feature Toggles:** En proyectos complejos, el código incompleto o en desarrollo puede integrarse a **main** apagado bajo un interruptor de configuración (por ejemplo, ConfigCat), permitiendo despliegues seguros a producción sin exponer la funcionalidad al usuario final.

Criterio	GitFlow Tradicional	Trunk-Based Development (TBD)
Ramas de Trabajo	Larga duración (semanas/meses), múltiples ramas intermedias (develop, release).	Vida ultra corta (horas o 1 día), una sola rama central (main).
Frecuencia de Integración	Baja (al final del sprint o ciclo).	Alta (diariamente o varias veces por día).
Resolución de Conflictos	Compleja, manual y con alto riesgo (Merge Hell).	Sencilla y casi nula gracias a lotes de cambio pequeños (Small Batches).
Definición de Terminado (DoD)	Código probado en rama local o stage.	Código fusionado en main , validado por CI y desplegable de inmediato.
Sinergia con Scrum	El incremento solo se consolida al final del Sprint.	Cada commit/merge en main es un incremento potencial y continuo.

3. Arquitectura de Pipeline y Desacoplamiento de Jobs

Uno de los fallos más comunes al iniciar con GitHub Actions es consolidar todas las tareas en un único job (como `test_and_build`). Esta práctica presenta serios problemas de arquitectura:

- **Desperdicio de Recursos:** Construir una imagen Docker y subirla a un registro toma minutos y consume cuotas de ancho de banda y almacenamiento que no deberían gastarse en borradores o PRs en revisión.
- **Contaminación de Registros:** Si un PR compila una imagen con la etiqueta `latest`, sobrescribir la versión estable de producción antes de que el código sea si quiera aprobado.
- **Vulnerabilidades de Seguridad:** Permitir que los Pull Requests de ramas externas o colaboradores tengan permisos de escritura en GitHub Packages (GHCR) abre la puerta a inyección de código malicioso o alteración de imágenes maestras.

Regla de Oro de la Separación CI / CD:

Job 1 (CI - Test): Corre en *TODO* evento (push y pull_request). No requiere secretos elevados ni permisos especiales (`contents: read`).

Job 2 (CD - Build & Push): Solo corre cuando el evento es un `push` directo o merge a la rama `main` Y requiere que el Job 1 haya concluido exitosamente (`needs: test`). Aquí y solo aquí se asigna el permiso `packages: write`.

4. Fase 1: Reestructuración y Refactorización del Pipeline de CI/CD

A continuación, se detalla el proceso paso a paso ejecutado en el repositorio de prácticas para migrar el flujo existente hacia el estándar TBD, con sus respectivas evidencias de terminal y configuración.

Paso 1: Renombrar el archivo de Workflow con Git

En el repositorio del proyecto, el archivo de workflow original se llamaba `test_and_buil.yaml` (con un error tipográfico en el nombre). Se procedió a renombrarlo formalmente a `ci.yaml` utilizando `git mv`.

```
# Verificación de estado inicial
```

```
git status
```

```
# Renombrado formal respetando el Árbol de seguimiento de Git
```

```
git mv .github/workflows/test_and_buil.yaml .github/workflows/ci.yaml
```

```
# Confirmación del rename en el staging area
```

```
git status
```

Razón de ingeniería: El comando `git mv` permite que el motor de Git identifique explícitamente la operación como un cambio de nombre (*renamed*) y preserve todo el historial de cambios del archivo, en lugar de interpretarlo como el borrado de un archivo y la creación de uno no relacionado.

Paso 1 — Renombrar el archivo de workflow

Abre tu terminal en la carpeta del proyecto (`bd`) y corre:

```
bash

git status
git mv .github/workflows/test_and_buil.yaml .github/workflows/ci.
git status
```

El segundo `git status` debería mostrarte algo como:

```
renamed: .github/workflows/test_and_buil.yaml ->
.github/workflows/ci.yaml
```

No hagas commit todavía — en el siguiente paso vamos a reemplazar también el contenido del archivo, así que lo dejamos en un solo commit limpio.



Figura 1: Guía y comando para renombrar el archivo de workflow de GitHub Actions sin commitear aún.

Paso 2: Inspección del Archivo Actual

Antes de alterar la lógica del pipeline, se inspecciona el contenido existente mediante el comando `cat` para no romper parámetros preexistentes como el login al registro de contenedores GHCR.

```
cat .github/workflows/ci.yaml
```

Paso 2 — Ver el contenido actual de tu workflow

Antes de reemplazarlo, quiero ver qué tienes exactamente ahora mismo (para no romper algo que ya te funciona, como el login/push a GHCR que mencionaste en tus últimos commits). Corre:

```
bash  
cat .github/workflows/ci.yaml
```

Pégame aquí el resultado completo. Con eso te doy el YAML final ajustado (en vez del genérico que puse en el documento) respetando lo que ya tenías funcionando, y seguimos con:

3. Probar el workflow con un PR de prueba
4. Configurar branch protection
5. Conectar el deploy (Render) y el feature toggle (ConfigCat)

Figura 2: Instrucciones para inspeccionar el YAML actual y hoja de ruta del laboratorio.

```
JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
$ cat .github/workflows/ci.yaml

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

permissions:
  contents: read
  packages: write # <-- Permiso para subir a GHCR

jobs:
  test_and_build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run ruff linter
        run: |
```

Figura 3: Terminal de desarrollo mostrando la visualización del archivo de workflow original unificado.

Paso 3: División en dos Jobs (Test vs. Build & Push)

Se procedió a refactorizar completamente el archivo `.github/workflows/ci.yaml` para establecer la división modular de responsabilidades.

La corrección: dividir en dos jobs. `test` corre siempre (PR y push).
`build_and_push` corre **solo** cuando el push ya es a `main`.

Paso 2 (continuación) — Reemplaza el contenido del archivo

Abre `.github/workflows/ci.yaml` en VS Code y reemplázalo completo por esto (mantuve tus versiones exactas: ruff 0.8.1, checkout@v4, login-action@v4, build-push-action@v7):

Figura 4: Explicación de la corrección arquitectónica: job 'test' siempre, 'build_and_push' solo en main.

A continuación se presenta el código final del pipeline:

```
name: Test and Build
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
  pull_request:
```

```
    branches:
```

```
      - main
```

```
  workflow_dispatch:
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    permissions:
```

```
      contents: read
```

```
    steps:
```

```
      - name: Checkout repository
```

```
        uses: actions/checkout@v4
```

```
      - name: Set up Python 3.11
```

```
        uses: actions/setup-python@v5
```

```
        with:
```

```
          python-version: "3.11"
```

```
      - name: Install dependencies
```

```
        run: |
```

```
          python -m pip install --upgrade pip
```

```
    pip install -r requirements.txt
```

```
- name: Run ruff linter
```

```
  run: |
```

```
    pip install ruff==0.8.1
```

```
    ruff check .
```

```
- name: Run tests
```

```
  run: |
```

```
    pytest tests.py
```

```
build_and_push:
```

```
  needs: test
```

```
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
```

```
  runs-on: ubuntu-latest
```

```
  permissions:
```

```
    contents: read
```

```
    packages: write # <-- Permiso específico para subir imágenes a GHCR
```

```
steps:
```

```
- name: Checkout repository
```

```
uses: actions/checkout@v4
```

```
- name: Login to GitHub Container Registry
```

```
uses: docker/login-action@v4
```

```
with:
```

```
registry: ghcr.io
```

```
username: ${ github.actor }
```

```
password: ${ secrets.GITHUB_TOKEN }
```

```
- name: Build and push Docker image
```

```
uses: docker/build-push-action@v7
```

```
with:
```

```
context: .
```

```
push: true
```

```
tags: ghcr.io/${ github.repository }:latest
```

Qué cambió y por qué (para cuando te pregunten):

- `test_and_build` se dividió en `test` y `build_and_push`.
- `build_and_push` tiene `needs: test` → no corre si el test falla.
- `build_and_push` tiene `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` → **nunca** se ejecuta en un PR, solo cuando el código ya está fusionado en `main`.
- `permissions: packages: write` se movió solo al job que lo necesita (buena práctica de mínimo privilegio).

Figura 5: Resumen técnico de los cambios: división de jobs, dependencia needs, condición if y permisos de paquetes.

Paso 4: Preparación, Commit y Push a Main

Una vez realizados el cambio de nombre y la edición del contenido, se consolidaron ambas modificaciones en un único commit atómico:

```
# Preparar los cambios en el staging area
```

```
git add .github/workflows/ci.yaml
```

```
git status
```

```
# Crear el commit semántico
```

```
git commit -m "refactor(ci): separar job de test y build_and_push;  
build_and_push solo corre en push a main"
```

```
# Enviar los cambios a la rama principal remota
```

```
git push origin main
```

Ahora haz el commit

```
bash
```

```
git add .github/workflows/ci.yaml  
git status
```

Confirma que te muestre el rename + la modificación juntos, algo así:

```
renamed: .github/workflows/test_and_buil.yaml ->  
.github/workflows/ci.yaml
```

Figura 6: Indicación para confirmar el rename y la edición en conjunto antes del commit.

```
JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)  
$ git add .github/workflows/ci.yaml  
  
JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)  
$ git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
Changes to be committed:  
  (use "git restore --staged <file>..." to unstage)  
    renamed:   .github/workflows/test_and_buil.yaml -> .github/workflows/ci.yaml  
    new file:   index.html  
    modified:   requirements.txt  
    new file:   script.js  
    new file:   seed-devops  
    new file:   style.css
```

Figura 7: Consola evidenciando el rename de 'test_and_buil.yaml' a 'ci.yaml' y archivos preparados en el stage.

```

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
● $ git commit -m "refactor(ci): separar job de test y build_and_push; build_and_push solo corre
[main 3a5c03b] refactor(ci): separar job de test y build_and_push; build_and_push solo corre e
6 files changed, 64 insertions(+), 6 deletions(-)
rename .github/workflows/{test_and_buil.yaml => ci.yaml} (75%)
create mode 100644 index.html
create mode 100644 script.js
create mode 160000 seed-devops
create mode 100644 style.css

```

Figura 8: Ejecución del commit atómico con 6 archivos modificados y 1 renombrado con 75% de similitud.

```

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
● $ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 1.78 KiB | 260.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/juanpablogarcia2020/bd.git
41a0d09..3a5c03b  main -> main

```

Figura 9: Envío exitoso del commit refactorizado hacia el repositorio remoto en GitHub.

5. Fase 2: Configuración de Gobernanza con GitHub Rulesets

En Trunk-Based Development, para asegurar que la rama `main` se mantenga siempre verde, limpia y desplegable, es imprescindible protegerla contra commits directos y exigir que pase por el filtro de revisión y pruebas.

Paso 3 — Branch Protection en tu repo real

Ve a: <https://github.com/juanpablogarcia2020/bd/settings/branches>

1. Clic en **Add branch protection rule** (o "Add rule").
2. En **Branch name pattern** escribe: `main`
3. Activa estas casillas:
 - ☒ **Require a pull request before merging**
 - Dentro de esta opción, pon **Required number of approvals before merging = 1**
 - ☒ **Require status checks to pass before merging**
 - Debajo aparecerá un buscador de checks — busca y selecciona `test` (así se llama tu job). Si no aparece todavía, dale a refrescar la página; a veces tarda un par de minutos en indexarse tras la primera ejecución.
 - Marca también **Require branches to be up to date before merging**
4. Baja hasta el final y dale **Create** (o **Save changes**).

Figura 10: Instrucciones para la configuración de las reglas de Branch Protection en el repositorio GitHub.

Configuración del Ruleset en GitHub

A través de la sección **Settings > Rules > Rulesets** se creará una nueva regla de protección para la rama `main`:

- **Ruleset Name:** `main`
- **Enforcement status:** `Active`
- **Target branches:** Añadir target > `Include default branch` (o por patrón `main`).

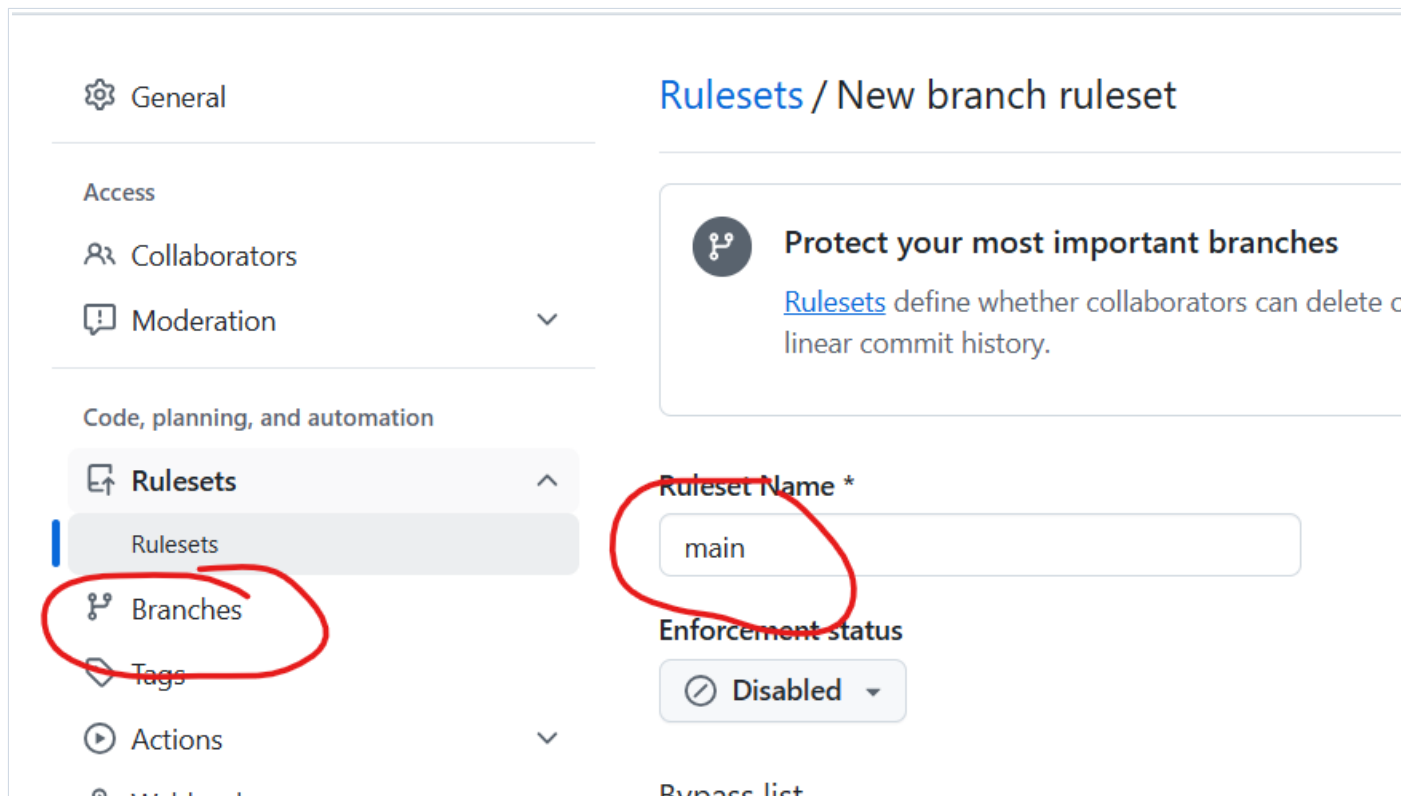


Figura 11: Interfaz de GitHub Rulesets con el nombre asignado 'main'.

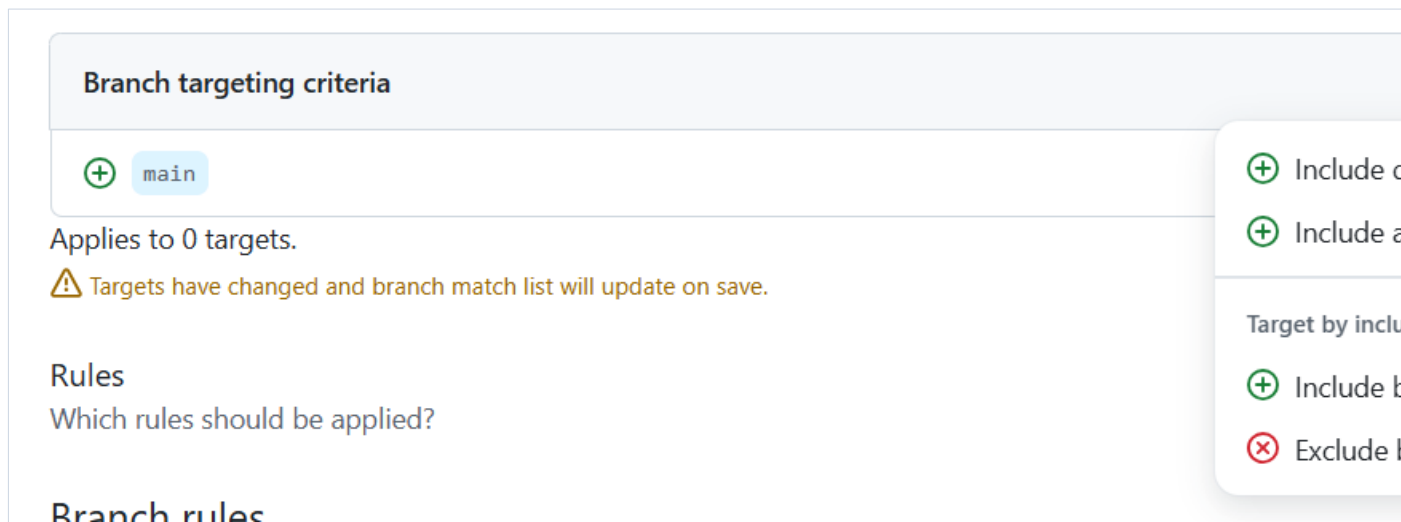


Figura 14: Criterio de selección de ramas objetivo asignado a la rama por defecto 'main'.

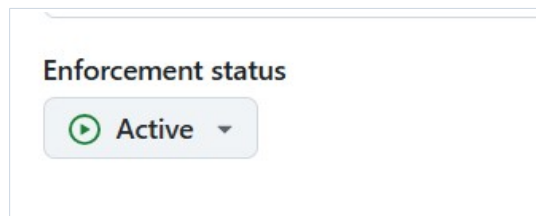


Figura 15: Activación formal del estado de cumplimiento (Enforcement status: Active).

Reglas de Pull Request y Status Checks Requeridos

Se activaron las tres políticas fundamentales de calidad:

1. **Require a pull request before merging:** Impide que cualquier usuario realice un `git push origin main` directo. Se configuró **Required approvals = 1**.
2. **Require status checks to pass before merging:** Obliga a que los tests automáticos concluyan con éxito. Se indexó y seleccionó el check `test` proveniente de GitHub Actions. Además, se marcó la casilla *Require branches to be up to date before merging* para prevenir desactualizaciones.
3. **Block force pushes:** Prohíbe terminantemente la ejecución de `git push --force` sobre `main`, salvaguardando la integridad histórica del proyecto.

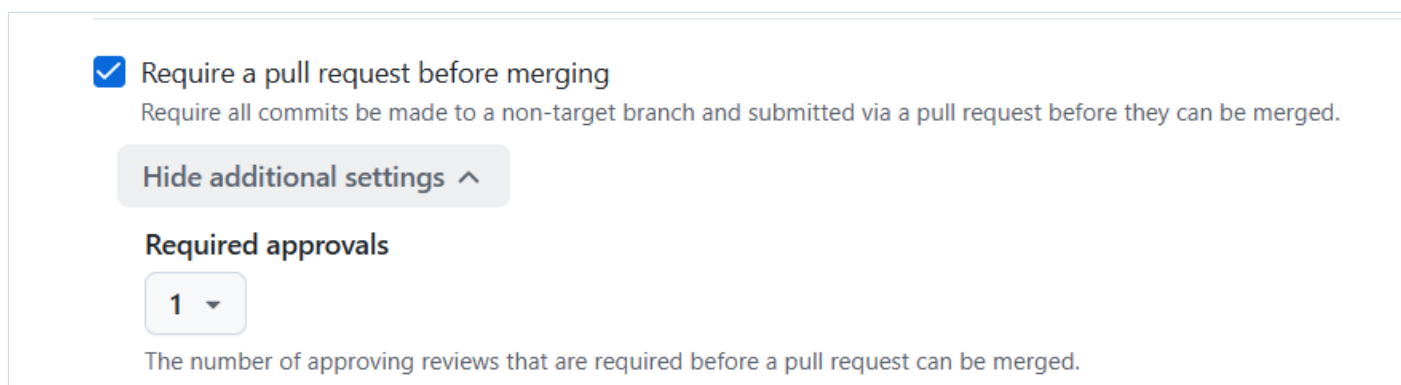


Figura 12: Configuración de la obligación de Pull Request y 1 aprobación antes de la fusión.

☒ **Require status checks to pass**
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another branch.

Hide additional settings ^

☒ **Require branches to be up to date before merging**
Whether pull requests targeting a matching branch must be tested with the latest code. This setting will not take effect until the setting is enabled.

☐ **Do not require status checks on creation**
Allow repositories and branches to be created if a check would otherwise prohibit it.

Status checks that are required

test

☒ **Block force pushes**
Prevent users with push access from force pushing to refs.

Figura 13: Configuración de status checks obligatorios asociando el job 'test' y bloqueo de force pushes.

Gestión de Excepciones (Bypass List) y Escenario Unipersonal

En proyectos donde un estudiante o desarrollador trabaja solo en su repositorio, activar una regla que exija 1 aprobación de un tercero crea un bloqueo insalvable (ya que GitHub no permite que el propio autor apruebe su propio Pull Request).

Para solucionar esto sin deshabilitar la regla, GitHub Rulesets ofrece la **Bypass list**:

- Se agregó el rol **Repository admin** a la lista de excepciones (Bypass list).
- Esto permite que el Ruleset imponga la obligación de crear el PR y esperar a que el CI pase en verde, mientras que el administrador puede proceder con el merge de forma controlada y auditada cuando no hay otros revisores disponibles.

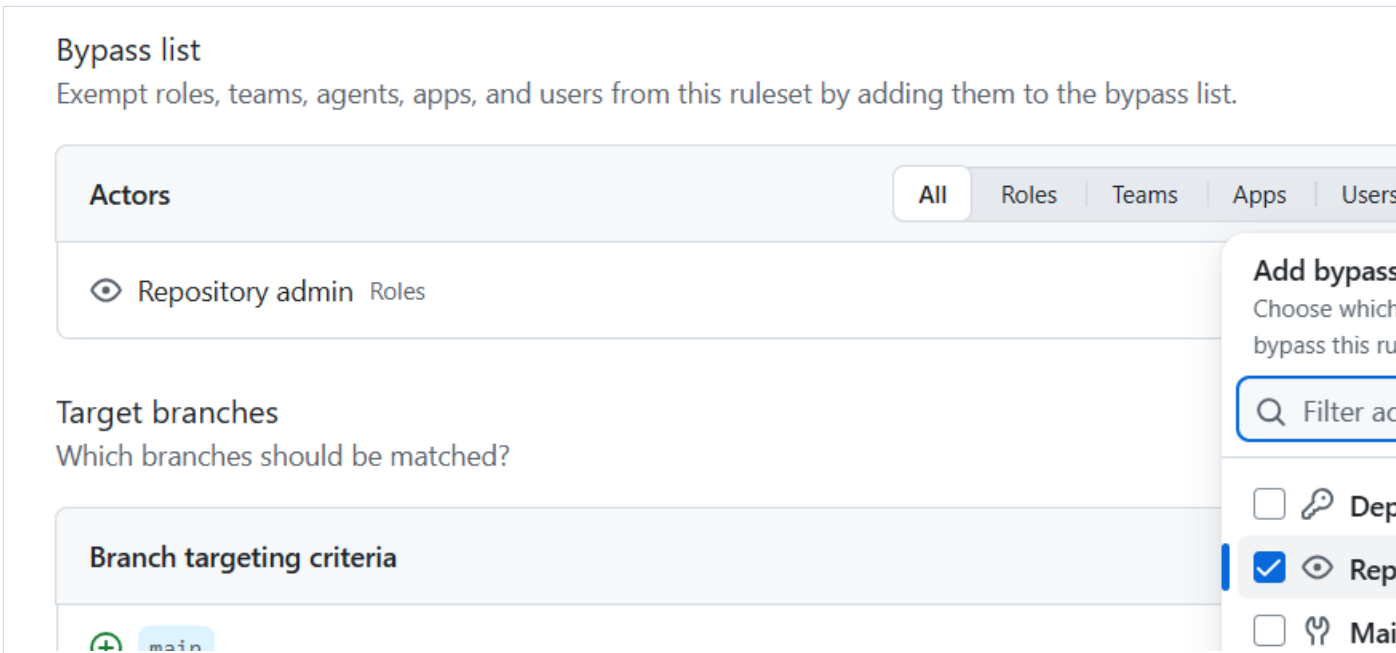


Figura 16: Asignación del rol Repository admin en la lista de excepciones (Bypass list).

Un detalle importante para tu caso

Como eres el único colaborador del repo, por defecto **como administrador podrías saltarte estas reglas**. Hay una casilla llamada **"Do not allow bypassing the above settings"** (a veces aparece como "Include administrators"). Te la dejo como decisión tuya:

- Si la **activas**: vivirás la restricción real — ni siquiera tú podrás mergear sin aprobación ni sin que pase el CI. Es lo más fiel al espíritu de TBD, pero como estás solo, vas a necesitar aprobar tus propios PRs desde otra cuenta o usar la opción de "Request changes/Approve" con un colaborador invitado, o desactivarla puntualmente si te trabas.
- Si la **dejas desactivada**: el CI sí bloqueará el merge para todos, pero tú como admin puedes forzar el merge si hace falta (útil para no bloquearte solo en la práctica).

Figura 17: Análisis técnico sobre el bypass de administrador en repositorios unipersonales vs. en equipo.

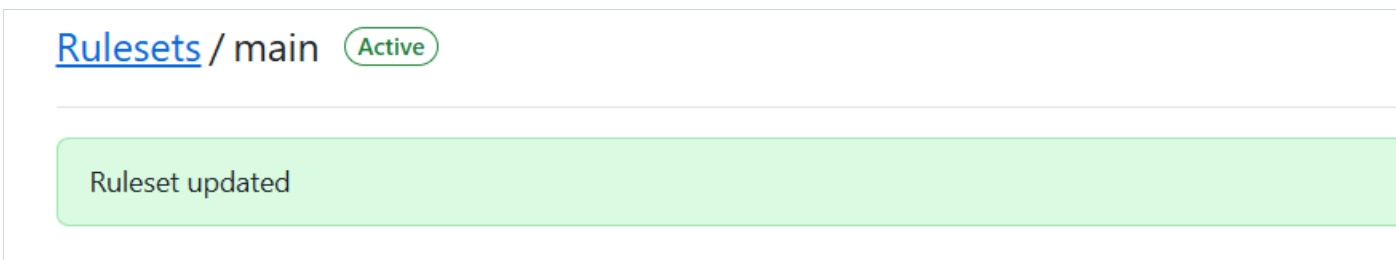


Figura 18: Mensaje de confirmación en color verde tras guardar el Ruleset exitosamente.

6. Fase 3: Verificación del Ciclo TBD con Pull Request en Vivo

Para poner a prueba la efectividad de las protecciones configuradas y comprobar el comportamiento del pipeline desacoplado, se ejecutará un ciclo de desarrollo real siguiendo la metodología Trunk-Based Development.

Paso 5: Creación de Rama de Vida Corta y Cambio Atómico

Se creará una rama efímera llamada `test/verificar-branch-protection`, se aplicará un cambio mínimo de prueba en `main.py` y se empujará al repositorio remoto:

```
# Creación y posicionamiento en la nueva rama
```

```
git checkout -b test/verificar-branch-protection
```

```
# Modificación mínima de prueba
```

```
echo "# prueba de branch protection" >> main.py
```

```
# Confirmación y push
```

```
git add main.py
```

```
git commit -m "test: verificar que el ruleset bloquea merge directo"
```

```
git push origin test/verificar-branch-protection
```

Paso 3.1 — Probar el Ruleset con un PR real

En tu terminal:

```
bash
```

```
git checkout -b test/verificar-branch-protection
```

Haz un cambio mínimo, por ejemplo un comentario en `main.py`:

```
bash
```

```
echo "# prueba de branch protection" >> main.py
```

```
git add main.py
```

```
git commit -m "test: verificar que el ruleset bloquea merge directo"
```

```
git push origin test/verificar-branch-protection
```

Figura 19: Guía paso a paso para crear la rama de vida corta y simular el cambio atómico.

```

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
$ git checkout -b test/verificar-branch-protection
Switched to a new branch 'test/verificar-branch-protection'

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (test/verificar-branch-protection)
$ echo "# prueba de branch protection" >> main.py

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (test/verificar-branch-protection)
$ git add main.py
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (test/verificar-branch-protection)
$ git commit -m "test: verificar que el ruleset bloquea merge directo"
[test/verificar-branch-protection 664aeab] test: verificar que el ruleset bloquea merge directo
1 file changed, 2 insertions(+), 1 deletion(-)

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (test/verificar-branch-protection)
$ git push origin test/verificar-branch-protection
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 396 bytes | 396.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'test/verificar-branch-protection' on GitHub by visiting:
remote:   https://github.com/juanpablogarcia2020/bd/pull/new/test/verificar-branch-protection
remote:
To https://github.com/juanpablogarcia2020/bd.git
 * [new branch]      test/verificar-branch-protection -> test/verificar-branch-protection

```

Figura 20: Consola de terminal ejecutando el checkout, la modificación del archivo, el commit y el push.

Paso 6: Apertura del Pull Request y Verificación de Bloqueos

Al ingresar a GitHub, la plataforma reconocerá inmediatamente la rama recién subida y habilitará la opción de abrir el Pull Request correspondiente.

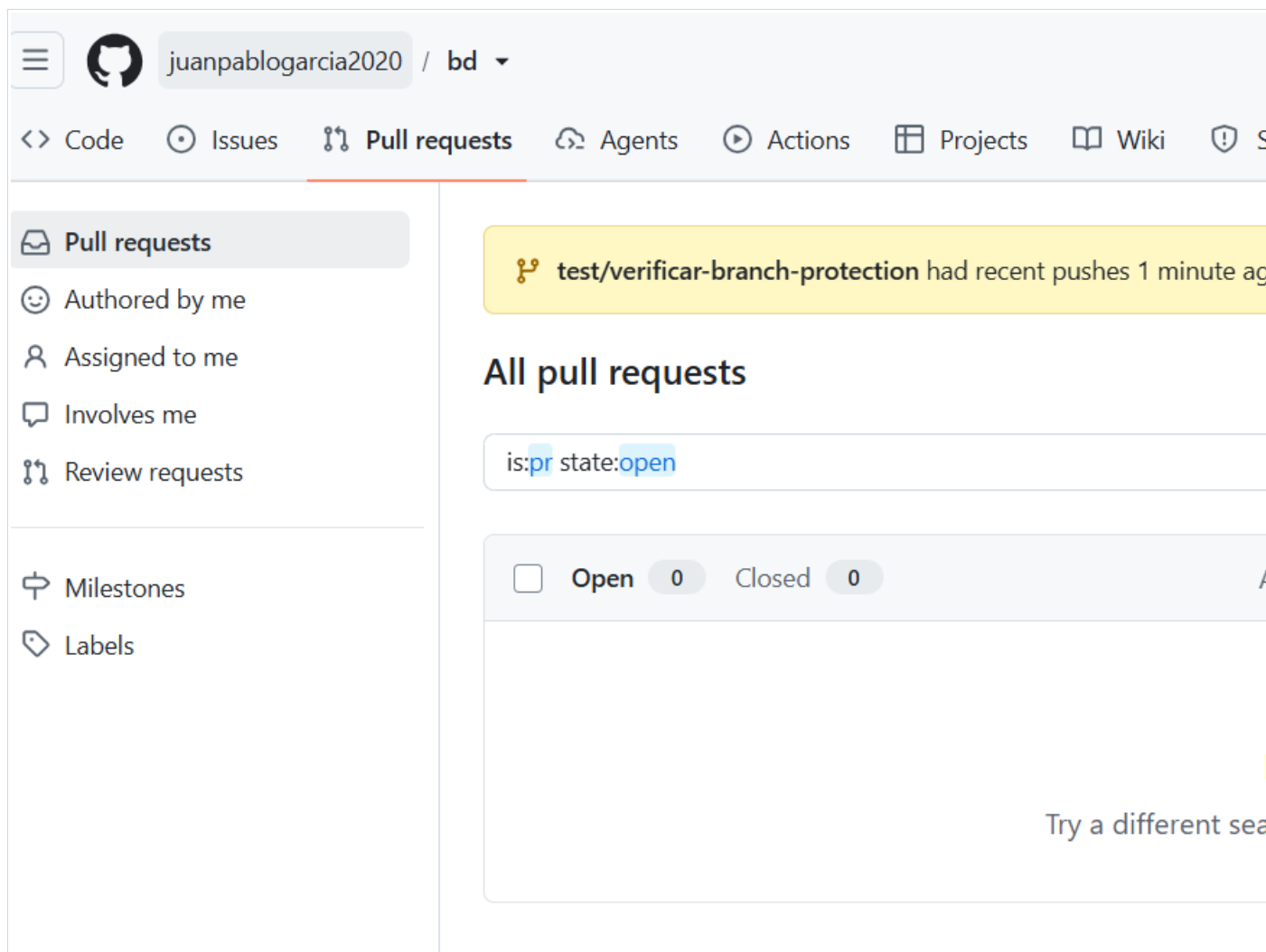


Figura 21: Alerta en GitHub invitando a abrir el Pull Request ('Compare & pull request').



Commits on Sep 18, 2026

test: verificar que el ruleset bloquea merge directo



juanpablogarcia2020 committed 6 minutes ago

Showing 1 changed file with 2 additions and 1 deletion.

```
3 main.py
... @@ -1,5 +1,6 @@
1 -
1 + #se buscan cerditos felices
2 2
3 3     class Calculator:
4 4         def suma(self, a: int, b: int) -> int:
5 5             return a+b
6 + # prueba de branch protection
```

Figura 22: Inspección del diff visual del commit confirmando la adición del comentario en main.py.

Una vez abierto el Pull Request #1, se validaron dos comportamientos críticos:

1. **El Ruleset bloquea el botón de merge directo:** La interfaz muestra la alerta roja "Merging is blocked - At least 1 approving review is required by reviewers with write access".
2. **El pipeline de CI se activa de forma diferenciada:**
 - El job `test` corre y termina en verde exitosamente en tan solo 10 segundos (marcado como `Required`).
 - El job `build_and_push` se marca como `Skipped` (omitido), demostrando fehacientemente que la condición `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` evita la compilación de la imagen Docker en la fase de revisión.

test: verificar que el ruleset bloquea merge directo

Open

juanpablogarcia2020 wants to merge 1 commit into main from test/verificar-branch-protection

Conversation 0

Commits 1

Checks 0

Files changed 1

juanpablogarcia2020 commented 1 minute ago

No description provided.

test: verificar que el ruleset bloquea merge directo

Review required

At least 1 approving review is required by reviewers with write access.

All checks have passed

1 skipped, 1 successful checks

1 skipped check

Test and Build / build_and_push (pull_request) Skipped 1 minute ago

1 successful check

Test and Build / test (pull_request) Successful in 10s

Merging is blocked

At least 1 approving review is required by reviewers with write access.

☐

Merge without waiting for requirements to be met (bypass rules)

Figura 23: Evidencia real del Pull Request #1: bot de merge bloqueado, check 'test' aprobado y 'build_and_push' omitido.

Paso 7: Fusión Controlada (Squash and Merge) y Activación de CD

Haciendo uso justificado del permiso de excepción por rol de administrador, se activó la opción "Merge without waiting for requirements to be met (bypass rules)" y se seleccionó la modalidad **Squash**

and merge.

RazÃ³n de ingenierÃa de Squash and Merge: En Trunk-Based Development, squash & merge unifica todos los micro-commits de la rama efÃmera en un Ãnico commit limpio y semÃntico en `main`, manteniendo un grafo de Git completamente lineal y fÃcil de revertir o auditar con `git bisect` si se requiere.

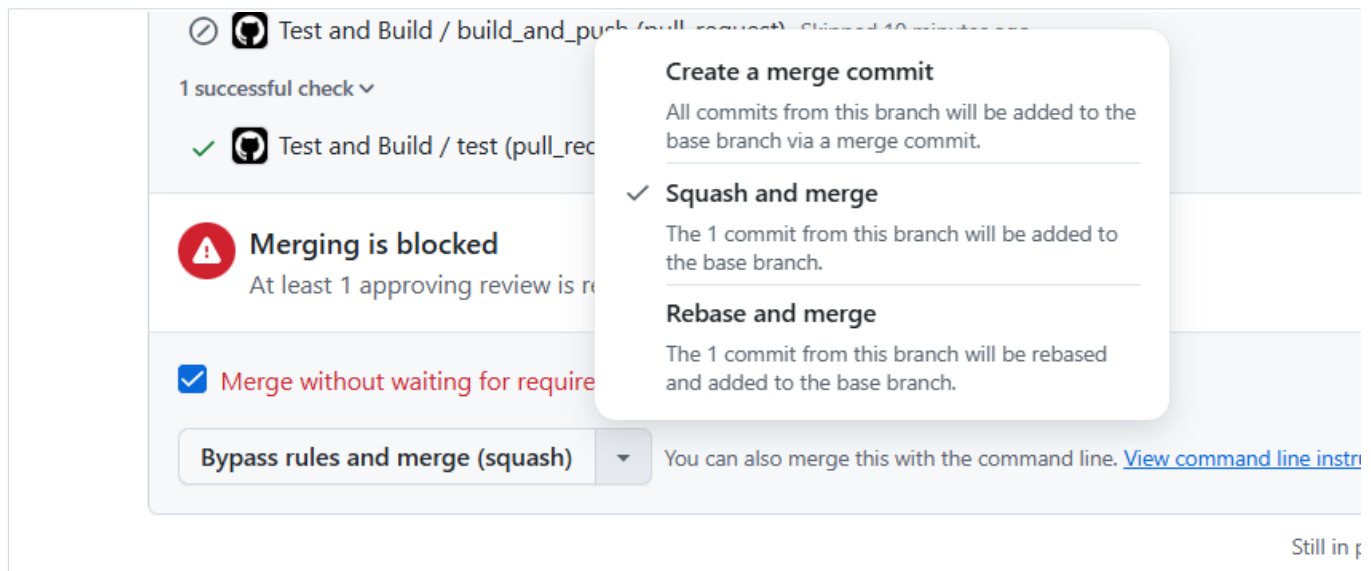


Figura 24: MenÃº desplegable seleccionando la estrategia 'Squash and merge' con bypass de reglas.

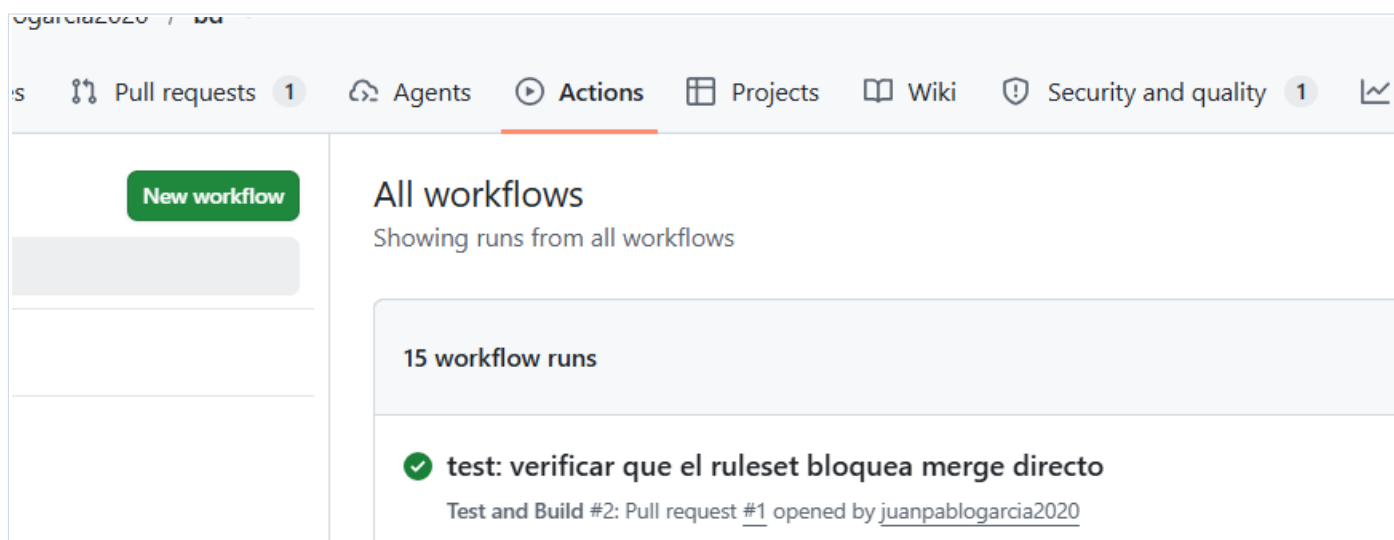


Figura 25: Registro en la pestaÃ±a Actions confirmando la ejecuciÃ³n exitosa de los tests para el PR #1.

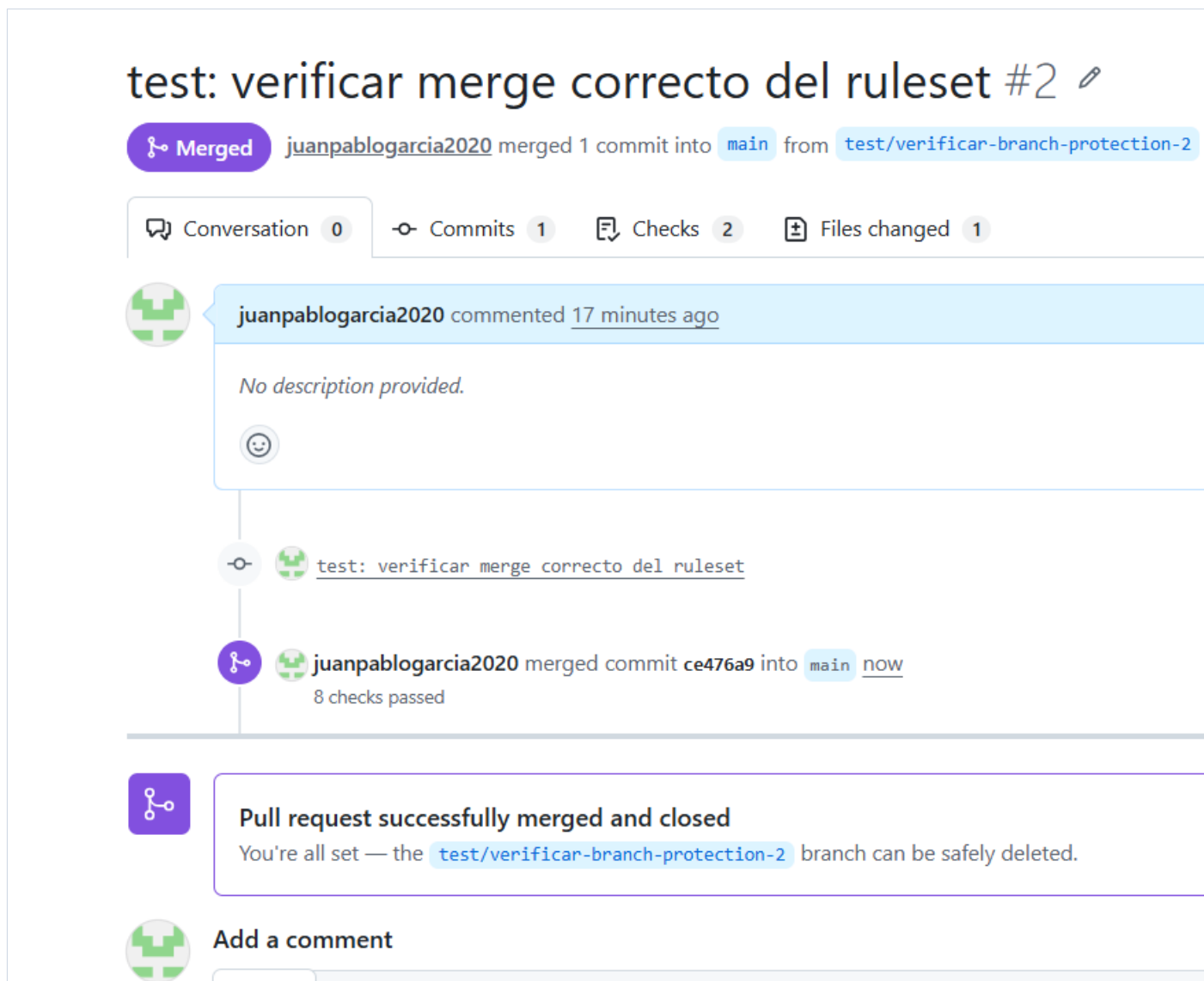


Figura 26: Confirmación en color violeta: 'Pull request successfully merged and closed'.

Inmediatamente tras el merge, el evento resultante en GitHub fue un **push** hacia **main**. Al consultar la pestaña **Actions**, se evidencia cómo el workflow se disparó automáticamente, y al cumplirse ahora la condición de rama **main**, procedió con la ejecución completa, construyendo y publicando la imagen Docker en GitHub Container Registry:

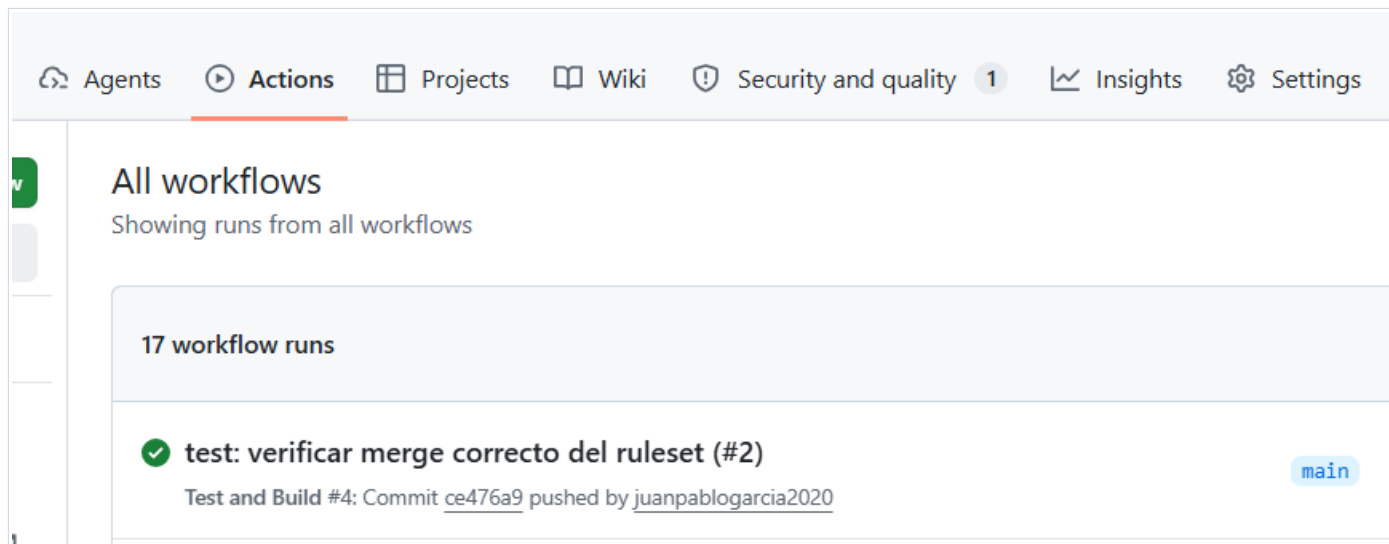


Figura 27: Ejecución exitosa del pipeline completo en main tras el merge, incluyendo el empaquetado del contenedor.

Paso 8: Higiene de Ramas (Branch Hygiene) y Limpieza

Un aspecto técnico fundamental es el orden estricto en la eliminación de ramas. En la práctica, **NUNCA se debe borrar una rama local o remota antes de verificar que en GitHub aparezca como 'Merged'**. Si se borra la rama prematuramente y el PR fue cerrado sin merge o requiere correcciones, los cambios y referencias pueden perderse o dificultar la trazabilidad.

Una vez confirmado el merge, se regresó a `main` en local, se descargaron los cambios consolidados y se eliminaron las ramas efímeras:

```
# Cambiar a la rama principal y sincronizar el nuevo commit de main

git checkout main

git pull origin main


# Eliminar la rama local que ya fue fusionada
```

```
git branch -D test/verificar-branch-protection-2
```

```
# Eliminar la rama en el repositorio remoto
```

```
git push origin --delete test/verificar-branch-protection-2
```

bash

```
git checkout main
git pull origin main
git branch -D test/verificar-branch-protection-2
git push origin --delete test/verificar-branch-protection-2
```

Figura 28: Comandos estándar para la sincronización de main y poda de ramas de trabajo.

```
JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (test/verificar-branch-protecti
● $ git checkout main
Switched to branch 'main'
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
● $ git pull origin main
From https://github.com/juanpablogarcia2020/bd
 * branch          main          -> FETCH_HEAD
Updating 3a5c03b..ce476a9
Fast-forward
 main.py | 1 +
 1 file changed, 1 insertion(+)

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
● $ git branch -D test/verificar-branch-protection-2
Deleted branch test/verificar-branch-protection-2 (was fb620ef).

JUAN PABLO@JUAN MINGW64 ~/Downloads/base de datos 8/corte 1/bd (main)
● $ git push origin --delete test/verificar-branch-protection-2
To https://github.com/juanpablogarcia2020/bd.git
 - [deleted]          test/verificar-branch-protection-2
```

Figura 29: Consola de terminal ejecutando el pull de main y la eliminación de la rama tanto en local como en remoto.

Paso 3 — Cerrado con evidencia real

Con esto tienes:

-  Workflow de CI separado en `test` y `build_and_push`
-  Ruleset activo en `main` (bloqueó merge sin aprobación)
-  Comportamiento verificado en vivo: PR bloqueado → bypass documentado → merge confirmado → push real a `main` → `build_and_push` disparado correctamente
-  Aprendiste (y puedes explicar) por qué nunca se borra una rama antes de confirmar "Merged"

Figura 30: Checklist final del laboratorio confirmando todos los hitos de Trunk-Based Development alcanzados.

7. Mapeo y Aplicación al Proyecto Real (SecurityMZManager)

Toda la metodología validada en el laboratorio de pruebas es directamente aplicable y representa el estándar operativo implementado en el proyecto **SecurityMZManager** (`MunGZar/SecurityMZManager`).

En **SecurityMZManager**, una solución integral para gestión de seguridad electrónica y cotizaciones, el archivo de pipeline `.github/workflows/ci-cd.yml` implementa con exactitud los mismos principios de arquitectura desacoplada:

```
# Estructura del Pipeline CI/CD en SecurityMZManager
```

```
jobs:
```

```
  backend:
```

```
    name: Backend (Test & Build)
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- uses: actions/checkout@v7
```

```
- name: Pruebas unitarias Jest y Build NestJS
```

```
run: |
```

```
npm ci
```

```
npx prisma generate
```

```
npm run test -- --passWithNoTests
```

```
npm run build
```

```
frontend:
```

```
name: Frontend (Build & Types)
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- uses: actions/checkout@v7
```

```
- name: Build Next.js con Turbopack y TypeScript Check
```

```
run: |
```

```
npm ci
```

```
npm run build

publish-packages:

  name: Publicar Packages en GitHub (GHCRC)

  needs: [backend, frontend]

  runs-on: ubuntu-latest

  if: github.ref == 'refs/heads/main' && (github.event_name == 'push' ||
github.event_name == 'workflow_dispatch')

  permissions:

    contents: read

    packages: write

  steps:

    - name: Docker build and push a ghcr.io/mungzar/securitymzmanager/...
```

Beneficios de TBD para el Equipo de SecurityMZManager

- **Integración sin Fricción:** Cada módulo desarrollado (ej. catálogo de productos Dahua/Imou, cotizaciones, gestión de clientes) se integra mediante micro-ramas en lugar de ramas eternas de características.
- **Confianza de Despliegue:** La rama `main` siempre está verde, permitiendo publicar paquetes Docker en GHCR con total garantía de que backend y frontend compilan y superan las pruebas unitarias.
- **Sintonía con Scrum:** Las historias de usuario se slicean (rebanan) en incrementos mínimos

entregables de 1 día, permitiendo que la Daily Scrum responda: "¿Qué voy a integrar hoy a main y qué necesito para mantener el pipeline en verde?".

8. Conclusiones y Buenas Prácticas de Ingeniería

1. **Eliminación del 'Merge Hell':** La adopción de Trunk-Based Development combinada con lotes pequeños de código reduce drásticamente los conflictos de combinación y acelera el tiempo de entrega de valor al usuario.
2. **DevSecOps y Mínimo Privilegio:** Limitar el token de GitHub Actions a solo lectura (`contents: read`) por defecto y aislar los permisos de escritura (`packages: write`) únicamente al job de publicación en `main` protege la cadena de suministro de software de ataques de inyección o contaminación de artefactos.
3. **Automatización del Definition of Done:** El pipeline de CI y los Rulesets de GitHub convierten la Definition of Done técnica en una garantía automática, objetiva e inalterable, donde ningún código sin pruebas o con linter roto puede tocar la rama productiva.
4. **Higiene de Control de Versiones:** La disciplina de esperar la confirmación de fusión antes de eliminar ramas y el uso de *Squash and Merge* garantizan un historial de Git limpio, lineal, legible y totalmente auditable.